COMP47910 Secure Software Engineering

Assignment 2 - Vulnerability Assessment

Summer 2026

Name: Kat Winter

Student Number: 25275037

Plagiarism: the unacknowledged inclusion of another person's writings or ideas or works, in any formally presented work (including essays, examinations, projects, laboratory reports or presentations). The penalties associated with plagiarism are designed to impose sanctions that reflect the seriousness of University's commitment to academic integrity.

Ensure that you have read the University's Briefing for Students on Academic Integrity and Plagiarism and the UCD Plagiarism Statement, Plagiarism Policy and Procedures, (http://www.ucd.ie/registrar/)

Declaration of Authorship

I declare that all material in this journal is my own work except where there is clear acknowledgement and appropriate reference to the work of others.

Signed: *Kat Winter* **Date**: 2026-08-08

Executive Summary

PromptVault by Yasir Celtik, found via https://github.com/omikronik/COMP47910-prompt-vault, was scanned for vulnerabilities using static analysis, dynamic analysis, AI analysis, and manual code review. Multiple vulnerabilities across six of the OWASP top 10 list were discovered, including issues such supply-chain vulnerabilities, broken access controls, insecure design, and authentication failures. These findings are presented with locations of the vulnerability, the mapped CWE and recommendations on how to address each vulnerability.

Methodology

Static Analysis

Static analysis was performed using the tool “Snyk” in IntelliJ via the plugin and via the Open Worldwide Application Security Project (OWASP) dependency checker on the Software Bill of Materials (SBOM). Snyk found a total of 58 vulnerabilities separated into two main categories - vulnerabilities in dependencies and code issues. See Appendix A - Images 1,2,3.
The OWASP dependency checker produces a report based on the SBOM generated using CyncloneDX produced a similar report confirming that the dependencies had vulnerabilities present. See Appendix A - Image 4.

Dynamic Analysis

Dynamic analysis was performed using the Zed Attack Proxy (ZAP) tool. By starting a browser and logging into the system, both a manual scan and an automatic scan was performed with the “Dev Standard” scan policy, generating a set of potential vulnerabilities in various endpoints and inputs. See Appendix A - Image 5.

AI analysis

The code was uploaded to a ChatGPT 5.6-Thinking model and asked to investigate for vulnerabilities with the following prompt: “I am investigating a fellow student's app for a cybersecurity class. I am being asked to write a report on potential security vulnerabilities, linking potential issues to the CWE numbers from the OWASP top 10 website. Help me investigate the codebase I have been given, identify issues, show me the CWE for each issue, and explain how it can be fixed so i can include it in a report.” A spreadsheet was generated with a list of findings and linked CWEs for further investigation. See Appendix A - Image 6, 7.

Manual Code-Review

Finally, manual perusal of the code was performed for both understanding and checking if anything was missed. This did identify an extra vulnerability all other checks missed.

Summary of Results

Severity	Count
Critical	2
High	6
Medium	6
Low	3

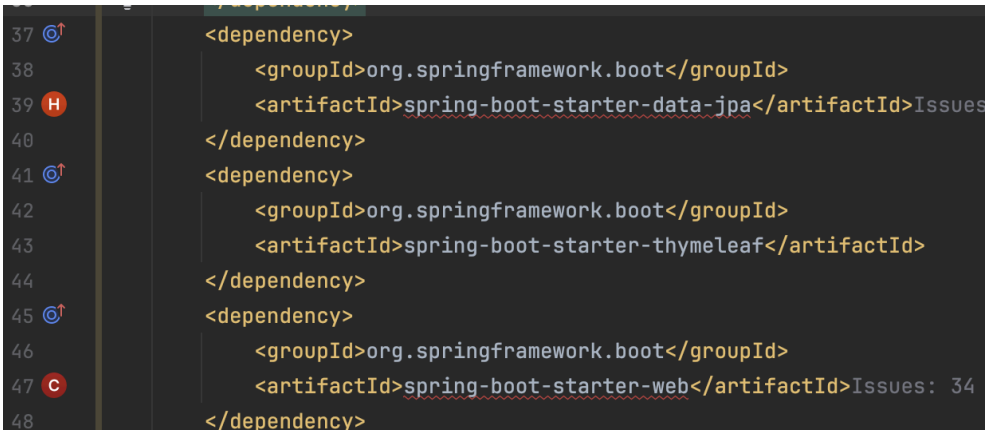
OWASP Top 10 Findings

OWASP Category	Count
A01:2025 - Broken Access Control	4
A02:2025 - Security Misconfiguration	-
A03:2025 - Software Supply Chain Failures	1
A04:2025 - Cryptographic Failures	-
A05:2025 - Injection	-
A06:2025 - Insecure Design	4
A07:2025 - Authentication Failures	4
A08:2025 - Software or Data Integrity Failures	-
A09:2025 - Security Logging & Alerting Failures	2
A10:2025 - Mishandling of Exceptional Conditions	1

Vulnerability Findings

Vulnerability Findings - Critical

Criticality	Critical
Description	User passwords are saved in plain-text to the database, and retrieved and compared plain-text in memory.
OWASP	A06:2025 – Insecure Design
CWE	CWE-256: Plaintext Storage of a Password
Location	AuthService.java:22-24, 36-46 `public Optional<User> login(LoginRequestDto request) { return userRepository.findByEmail(request.email()) .filter(u -> u.getPassword().equals(request.password())) .filter(User::isActive); }` `User user = User.builder() .firstName(request.firstName()) .lastName(request.lastName()) .username(request.username()) .email(request.email()) .password(request.password()) .role(UserRole.USER) .active(true) .build();` `userRepository.save(user);`
Exploit	A malicious actor obtaining a database dump will be able to view and retrieve user passwords. If a malicious actor obtains access to the server's memory, as users are retrieved from the database and the passwords are compared in plain-text, even an incorrect login attempt retrieves the password and will be available in memory for theft.
Recommended Fix	A cryptographically secure password encoder such as BCryptPasswordEncoder should be used and only the hashes of the passwords should be stored in the database. During login the provided password should be hashed using the same encoder and compared to the stored hash value. This way a database dump and memory access do not leak the user's actual password.
Source	AI analysis (A.6), Static Analysis - Snyk (A.3)

Criticality	Critical
Description	The application has four dependencies with critical vulnerabilities. It has many more with lower severity vulnerabilities as well.
OWASP	A03:2025 Software Supply Chain Failures
CWE	CWE-1395: Dependency on Vulnerable Third-Party Component
Location	pom.xml 
Exploit	Several critical CVEs (e.g. https://www.cve.org/CVERecord?id=CVE-2026-54512) have been published for the spring-boot-starter-web version that PromptVault is using. An attacker who becomes aware of the package version the server is running could use this information to exploit the server.
Recommended Fix	Dependencies should be upgraded regularly as fixes are released. In this case, the static analysis and dependency checker both recommend upgrading to org.springframework.boot:spring-boot-starter-web be upgraded to at least version 3.5.15 to fix these critical alerts. There are several other dependencies that have identifier issues during static analysis, so a general package upgrade should be performed to reduce dependencies on packages with known issues.
Source	Static Analysis - Snyk (A.1, A.2), OWASP Dependency Checker (A.4)

Vulnerability Findings - High

Criticality	High
Description	Any logged in user can view and edit another user's prompts by manipulating the ID in the edit url
OWASP	A01:2025 – Broken Access Control
CWE	CWE-639: Authorization Bypass Through User-Controlled Key
Location	PromptController.java `@GetMapping("/{id}/edit")` `public String editPromptPage(@PathVariable long id, Model model, HttpSession session) {` `  if (!sessionService.isLoggedIn(session)) {` `  return "redirect:/auth/login";` `  }` `  Prompt prompt = promptService.getPromptById(id).orElseThrow();` `  model.addAttribute("prompt", prompt);` `  model.addAttribute("categories", promptService.getAllCategories());` `  return "prompt/edit";` `}` `@PostMapping("/{id}/edit")` `public String editPrompt(@PathVariable long id, HttpSession session, @ModelAttribute CreatePromptRequestDto dto) {` `  if (!sessionService.isLoggedIn(session)) {` `  return "redirect:/auth/login";` `  }` `  User user = sessionService.getCurrentUser(session);` `  promptService.editPrompt(id, dto, user);` `  return "redirect:/prompts";` `}`
Exploit	1. Login as a user 2. Click "My Prompts" 3. Click "Edit" on any prompt 4. Change the ID in the url to any other ID. For example, {base_url}/prompts/4/edit -> {base_url}/prompts/1/edit 5. See another user's prompt 6. Change the prompt title/content/category/visibility 7. Click "Save Changes"

	8. Perform step 4 again 9. See the changes were persisted
Recommended Fix	Both the get and post endpoints for editing a prompt correctly check if a user is logged in but omit checking if the user owns the requested prompt. Both endpoints should include a check that when retrieving or editing the prompt is being done by the user who owns it.
Source	AI analysis (A.6), Manual Analysis

Criticality	High
Description	Any logged in user can submit messages to any other user's prompt conversation
OWASP	A01:2025 – Broken Access Control
CWE	CWE-639: Authorization Bypass Through User-Controlled Key
Location	ConversationController.java:58-66 `@PostMapping("/{id}/messages")` `public String sendMessage(@PathVariable Long id, @RequestParam String content, HttpSession session) {` `  User user = sessionService.getCurrentUser(session);` `  if (user == null)` `  return "redirect:/auth/login";` `  conversationService.sendMessage(id, content, user);` `  return "redirect:/conversations/" + id;` `}`
Exploit	1. Login as USER1 to obtain the JSESSIONID from the cookie. 2. Find a conversation that belongs to USER2 (by viewing the database or logging in as another user on another browser) 3. User curl to create a POST request with the following information, replacing the cookie and conversation_id: `curl --location 'http://localhost:8082/conversations/{USER2_CONVERSAION_ID}/messages' \` `--header 'Cookie: JSESSIONID={USER1_JSESSIONID} \` `--header 'Content-Type: application/x-www-form-urlencoded' \` `--data-urlencode 'content=This is sent by USER1 to USER2 Conversation'` 4. View the conversation for USER2, see that the message was posted successfully.

Recommended Fix	The conversation controller correctly checks that the user who wishes to view the conversation is the owner of the conversation. `if (conversation.getOwner().getId() != user.getId()) {` `  return "redirect:/dashboard";` `}` A similar guard should be implemented for this sendMessage endpoint.
Source	AI analysis (A.6), Manual Analysis

Criticality	High
Description	Any logged in user can create a conversation for another user by knowing their prompt IDs
OWASP	A01:2025 – Broken Access Control
CWE	CWE-639: Authorization Bypass Through User-Controlled Key
Location	ConversationController.java:29-37 `@PostMapping("/start")` `public String startConversation(@RequestParam(required = false) Long promptId, HttpSession session) {` `  User user = sessionService.getCurrentUser(session);` `  if (user == null)` `  return "redirect:/auth/login";` `  Conversation conversation = conversationService.createConversation(user, promptId);` `  return "redirect:/conversations/" + conversation.getId();` `}`
Exploit	1. Login as USER1 to obtain the JSESSIONID from the cookie. 2. Find a private prompt that belongs to USER2 (by viewing the database or logging in as another user on another browser) 3. User curl to create a POST request with the following information, replacing the cookie and conversation_id: `curl --location 'http://localhost:8082/conversations/start' \` `--header 'Cookie: JSESSIONID={USER1_JSESSIONID} \` `--header 'Content-Type: application/x-www-form-urlencoded' \` `--data-urlencode 'promptId={USER2_PRIVATE_PROMPT_ID}` 4. View the conversation started for USER1, using the private prompt of user2
Recommended	The conversation controller correctly checks that the user who wishes to

Fix	view the conversation is the owner of the conversation. `if (conversation.getOwner().getId() != user.getId()) {` `  return "redirect:/dashboard";` `}` A similar guard should be implemented for this startConversation endpoint and disallow starting conversations with prompts that do not belong to the user or are not shared.
Source	AI analysis (A.6), Manual Analysis

Criticality	High
Description	Sessions are established on login and are not refreshed or rechecked when accessing endpoints. When a user becomes disabled, they are prevented from logging in but an already logged in user retains access until they log out.
OWASP	A07:2025 Authentication Failures
CWE	CWE-613: Insufficient Session Expiration
Location	AuthController.java:33-50 `@PostMapping("/login")` `public String login(@ModelAttribute LoginRequestDto loginRequest,` `  HttpSession session,` `  RedirectAttributes redirectAttributes) {` `  log.info("Login attempt {}", loginRequest.email());` `  Optional<User> user = authService.login(loginRequest);` `  if (user.isEmpty()) {` `  redirectAttributes.addFlashAttribute("error", "Invalid email or password.");` `  log.info("Failed login attempt for {}", loginRequest.email());` `  return "redirect:/auth/login";` `  }` `  session.setAttribute("user", user.get());` `  log.info("Successful login attempt for {}", loginRequest.email());` `  return "redirect:/dashboard";` `}`
Exploit	1. Login as USER1 2. Login on another browser as admin user 3. As admin, set USER1 status to disabled 4. As USER1 continue to use the application

	5. As USER1 logout 6. Attempt to login with USER1, see that USER1 is disabled
Recommended Fix	A security config should be added that includes a SecurityFilterChain bean that performs session checking on all requests. This will ensure that a disabled user immediately loses access on the next request instead of on logout: `@Bean` `SecurityFilterChain securityFilterChain(HttpSecurity http) {` `  return http` `  .authorizeHttpRequests(registry -> {` `  registry.anyRequest().authenticated();` `  })` `  .build();` `}`
Source	AI analysis (A.6), Manual Analysis

Criticality	High
Description	There are no requirements when constructing a password
OWASP	A07:2025 Authentication Failures
CWE	CWE-521: Weak Password Requirements
Location	AuthService.java:33-50 `@PostMapping("/register")` `public String register(@ModelAttribute RegisterRequestDto registerRequest,` `  RedirectAttributes redirectAttributes) {` `  boolean success = authService.register(registerRequest);` `  if (!success) {` `  redirectAttributes.addFlashAttribute("error", "An account with that email or username already exists.");` `  return "redirect:/auth/register";` `  }` `  redirectAttributes.addFlashAttribute("success", "Account created. You can now sign in.");` `  return "redirect:/auth/login";` `}`
Exploit	A user can select anything as their password, even a single character. This could lead to their password being easily guessable.
Recommended Fix	Requirements for a password should be implemented, requiring a minimum length and use of special characters and numbers.

Source	Manual Analysis

Criticality	High
Description	Many forms are missing their anti-CSRF tokens.
OWASP	A07:2025 Authentication Failures
CWE	CWE-352: Cross-Site Request Forgery (CSRF)
Location	
Exploit	If a user is logged in and clicks a malicious link for one of the above endpoints, it will be performed as though they intended to submit it. This could allow an attacker to re-enabled their account, disabled other users' accounts, manipulate prompts/categories/etc.
Recommended Fix	Include the spring security package and each form should also then include the CSRF token to prevent this vulnerability. `<input th:name="${_csrf.parameterName}" th:value="${_csrf.token}" type="hidden"/>` https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html#csrf-integration-form Also include the Security Filter Chain bean: `@Bean SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception { return http // Do not call csrf(csrf -> csrf.disable()) .authorizeHttpRequests(auth -> auth .requestMatchers( "/auth/login", "/auth/register", "/css/**" ).permitAll()`

	<pre>.anyRequest().authenticated()) .formLogin(form -> form .loginPage("/auth/login") .permitAll()) .build(); }</pre>
Source	Static Analysis - Snyk (A.3), Dynamic Analysis - ZAP (A.5)

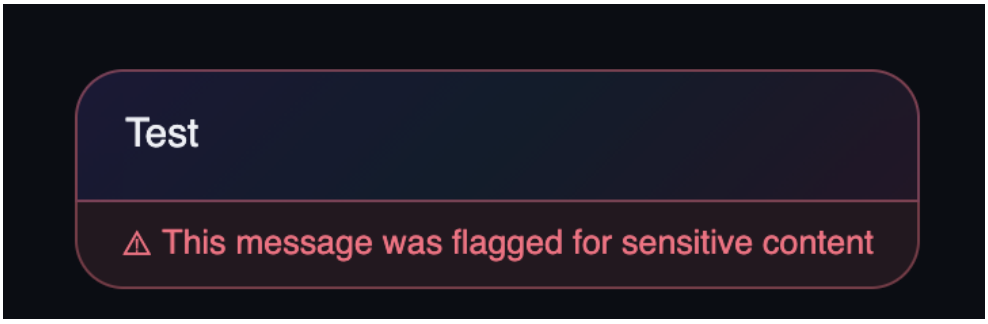
Vulnerability Findings - Medium

Criticality	Medium
Description	Many forms are missing their anti-CSRF tokens.
OWASP	A07:2025 Authentication Failures
CWE	CWE-307: Improper Restriction of Excessive Authentication
Location	Lack of SecurityConfig, AuthController.java:33-50 <pre>@PostMapping("/login") public String login(@ModelAttribute LoginRequestDto loginRequest, HttpSession session, RedirectAttributes redirectAttributes) { log.info("Login attempt {}", loginRequest.email()); Optional<User> user = authService.login(loginRequest); if (user.isEmpty()) { redirectAttributes.addFlashAttribute("error", "Invalid email or password."); log.info("Failed login attempt for {}", loginRequest.email()); return "redirect:/auth/login"; } session.setAttribute("user", user.get()); log.info("Successful login attempt for {}", loginRequest.email()); return "redirect:/dashboard"; }</pre>

Exploit	The login endpoint has no rate-limiting which allows an attacker to brute force guess user passwords. Using rainbow tables, dictionary attacks, or fuzzing allows a malicious actor to run as many requests as they want to try and guess user passwords.
Recommended Fix	Usage of a Bucket4j filter (https://www.baeldung.com/spring-bucket4j) can be added to the security config to rate limit users. A regular user for an application such as this likely doesn't require more than 2-5 requests per second. Even just rate-limiting the login endpoint will decrease the likelihood of a malicious actor being able to discover a user's password. `@Bean` `SecurityFilterChain securityFilterChain(` `HttpSecurity http,` `LoginRateLimitFilter loginRateLimitFilter)` `throws Exception {` `return http` `.addFilterBefore(` `loginRateLimitFilter, # <- Create and add filter` `UsernamePasswordAuthenticationFilter.class)` `.authorizeHttpRequests(auth -> auth` `.requestMatchers(` `"/auth/login",` `"/auth/register",` `"/css/**")` `.permitAll()` `.anyRequest()` `.authenticated())` `.build();` `}`
Source	AI Analysis (A.6), Manual Analysis

Criticality	Medium
Description	Many locations validate user input in the browser but perform no server side checks of the input.
OWASP	A06:2025 Insecure Design
CWE	CWE-602: Client-Side Enforcement of Server-Side Security
Location	Policy Keyword example: Enforced client side at policy-keywords/create.html:24-38 `<div class="card">` `<form method="post"` `th:action="@{/admin/policy-keywords/create}">` `<div class="form-group">` `<label for="content">Keyword</label>` `<input`

	 `` type="text" id="content" name="content" maxlength="255" required /> </div> <input type="submit" value="Add Keyword" /> </form> </div> ``` Not enforced server side: AdminPolicyKeywordController.java:55-62: ``` @PostMapping("/create") public String createKeyword(HttpSession session, @ModelAttribute CreatePolicyKeywordDto dto) { if (!isAdmin(session)) return "redirect:/dashboard"; User user = sessionService.getCurrentUser(session); policyKeywordService.createKeyword(dto, user); return "redirect:/admin/policy-keywords"; } ``` PolicyKeywordService.java:33-40 ``` @Transactional public void createKeyword(CreatePolicyKeywordDto dto, User user) { PolicyKeyword keyword = PolicyKeyword.builder() .content(dto.content()) .createdBy(user) .build(); policyKeywordRepository.save(keyword); log.info("created policy keyword '{}'", keyword.getContent()); } ``` PolicyKeyword.java:27-28 ``` @Column(nullable = false, unique = true, length = 255) private String content; ``` An empty string is not null so the `nullable` requirement isn't working as expected. Other locations include: AdminCategoryController.java, AdminPolicyKeywordController.java, AuthController.java, PromptController.java
Exploit	Login as an admin user to get the admin JSESSIONID from the cookie. Then execute the following curl request to create an empty policyKeyword. This shows as every prompt & conversation being flagged as every string in java contains the empty string: ``` curl --location 'http://localhost:8082/admin/policy-keywords/427/edit' \ --header 'Cookie: JSESSIONID={ADMIN_JSESSION_ID}' \ ```

	`--header 'Content-Type: application/x-www-form-urlencoded' \` `--data-urlencode 'content='` Result:  Can also create a user with no information: Execute the following curl request to create a user with no firstName, lastName, username, email, or password: `curl --location 'http://localhost:8082/auth/register' \` `--header 'Content-Type: application/x-www-form-urlencoded' \` `--data-urlencode 'firstName=' \` `--data-urlencode 'lastName=' \` `--data-urlencode 'username=' \` `--data-urlencode 'email=' \` `--data-urlencode 'password='`
Recommended Fix	All locations that accept user input should validate that input is in the correct format server side. For example, adding @NotBlank annotation to the models or explicitly checking that the values are not empty via `if (dto.content().isBlank())` `return "redirect:/dashboard";`
Source	AI Analysis (A.6), Manual Analysis

Criticality	Medium
Description	Sensitive user information such as emails are written to the logs
OWASP	A09:2025 Security Logging & Alerting Failures
CWE	CWE-532: Insertion of Sensitive Information into Log File
Location	AuthController.java:37 `log.info("Login attempt {}", loginRequest.email());` AuthController.java:43

	`log.info("Failed login attempt for {}", loginRequest.email());` AuthController.java:48 `log.info("Successful login attempt for {}", loginRequest.email());` UserService.java:34 `@Transactional` `public void setActive(Long id, boolean active) {` `  User user = userRepository.findById(id).orElseThrow();` `  user.setActive(active);` `  log.info("set user {} active={}", user.getEmail(), active);` `}`
Exploit	User emails are written directly to logs. If a malicious actor gains access to the logs they will have a record of emails of users who use PromptVault, as well as other information about the users accounts. For this service, this is likely benign but it could lead to extortion of these users with their information. It can also be a breach of GDPR as an email is likely considered PII.
Recommended Fix	Replace the user.getEmail() calls with user.getId() as used elsewhere in logging. This preserves the audit trail but prevents logging of PII to logs as well as preventing a malicious actor from easily identifying users from just gaining access to the logs.
Source	AI Analysis (A.6), Manual Analysis

Criticality	Medium
Description	Arbitrary user input is written to the logs without sanitsation
OWASP	A09:2025 Security Logging & Alerting Failures
CWE	CWE-117: Improper Output Neutralization for Logs
Location	AuthController.java:37 `log.info("Login attempt {}", loginRequest.email());` AuthController.java:43 `log.info("Failed login attempt for {}", loginRequest.email());` AuthController.java:48 `log.info("Successful login attempt for {}", loginRequest.email());` CategoryService.java:42 `log.info("created category '{}'", category.getName());` CategoryService.java:51 `log.info("edited category '{}'", category.getName());` ConversationService.java:71 `log.info("created conversation '{}' for user {}", saved.getTitle(), user.getId());` PromptService.java:97 `log.info("created prompt '{}' for user {}", prompt.getTitle(),`

	`user.getId());` PromptService.java:133 `log.info("edited prompt '{}' for user {}", prompt.getTitle(), user.getId());` UserService.java:34 `@Transactional` `public void setActive(Long id, boolean active) {` `  User user = userRepository.findById(id).orElseThrow();` `  user.setActive(active);` `  log.info("set user {} active={}", user.getEmail(), active);` `}`
Exploit	Using curl commands, an attacker can forge log entries. For example: `curl --location 'http://localhost:8082/auth/login' \` `--header 'Content-Type: application/x-www-form-urlencoded' \` `--data-urlencode $'email=admin@promptvaul.com\n2026-07-21 INFO Administrator login successful' \` `--data-urlencode 'password='` Shows up in the logs as: `2026-08-03 22:07:32.550 [http-nio-8082-exec-4] INFO c.y.p.controller.AuthController login - Login attempt admin@promptvaul.com` `2026-07-21 INFO Administrator login successful` `Hibernate: select u1_0.id,u1_0.active,u1_0.created_by,u1_0.created_on,u1_0.email,u1_0.first_name,u1_0.last_name,u1_0.login_attempts,u1_0.pas` `2026-08-03 22:07:32.565 [http-nio-8082-exec-4] INFO c.y.p.controller.AuthController login - Failed login attempt for admin@promptvaul.com` `2026-07-21 INFO Administrator login successful`
Recommended Fix	All input should be treated as potentially hostile. Input that will be written to logs should be sanitised before being written. For example, implement a log sanitiser: `public final class LogSanitizer {` `  private static final int MAX_LOG_VALUE_LENGTH = 200;` `  private LogSanitizer() {` `  }` `  public static String sanitize(String value) {` `  if (value == null) {` `  return "<null>";` `  }` `  String sanitized = value` `  .replace("\r", "\\r")` `  .replace("\n", "\\n")` `  .replace("\t", "\\t");` `  if (sanitized.length() > MAX_LOG_VALUE_LENGTH) {` `  return sanitized.substring(0, MAX_LOG_VALUE_LENGTH)` `  + "...[truncated]";` `  }` `  return sanitized;` ` }

	``}``` Then use as: ```log.info("Login attempt {}", LogSanitizer.sanitize(loginRequest.email()));```
Source	AI Analysis (A.6), Manual Analysis

Criticality	Medium
Description	HTTP responses do not include a Content-Security-Policy header.
OWASP	A06:2025 Insecure Design
CWE	None
Location	Lack of SecurityConfig Inspect any response and see that the Content-Security-Policy is missing from it.
Exploit	A lack of this header means that a malicious script could be loaded by a user and accidentally run. The header is not sufficient to avoid exploitation but is a recommended layer of security.
Recommended Fix	To the security Policy, add the content security policy headers: ```@Bean SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception { return http .headers(headers -> { headers.contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'; " + "script-src 'self'; " + "style-src 'self'; " + "img-src 'self' data:; " + "font-src 'self'; " + "connect-src 'self'; " + "object-src 'none'; " + "base-uri 'self'; " + "form-action 'self'; " + "frame-ancestors 'none'")); }) .build(); }```
Source	Dynamic Analysis - ZAP (A.5)

Criticality	Medium
Description	Anti-Clickjacking headers are missing
OWASP	A06:2025 Insecure Design
CWE	CWE-1021: Improper Restriction of Rendered UI Layers or Frames
Location	Lack of SecurityFilterChain and SecurityConfig
Exploit	A malicious actor can position a PromptVault frame under a decoy frame, causing the user to perform unintended actions in PromptVault without their knowledge.
Recommended Fix	Add the anti-clickjacking headers to the security config: `@Bean` `SecurityFilterChain securityFilterChain(HttpSecurity http)` `throws Exception {` `  return http` `  .headers(headers -> {` `  headers.contentSecurityPolicy(csp ->` `csp.policyDirectives(` `  "default-src 'self'; " +` `  "frame-ancestors 'none'"` `  ));` `  headers.frameOptions(frame -> frame.deny());` `  })` `  .build();` `}`
Source	Dynamic Analysis - ZAP (A.5)

Vulnerability Findings - Low

Criticality	Low
Description	The JSESSIONID cookie is issued without a SameSite Attribute
OWASP	A01:2025 Broken Access Control
CWE	CWE-1275: Sensitive Cookie with Improper SameSite Attribute
Location	application.properties is missing cookie settings
Exploit	A lack of the SameSite attribute can allow a malicious actor to attach the user's JSESSIONID cookie allowing actions to be performed without the user's express intention.
Recommended	Add the following to application.properties:

Fix	``` server.servlet.session.cookie.same-site=lax server.servlet.session.cookie.secure=true server.servlet.session.cookie.http-only=true ``` Before: Name | Value | Domain | Path | Expires / Max-Age | Size | HttpOnly | Secure | SameSite | Last Accessed | Partition Key JSESSIONID | 8C40AFAF42A4578E1B7FA59946691214 | localhost | / | Session | 42 | true | false | | Mon, 03 Aug 2026 17:04:31 GMT | After: Name | Value | Domain | Path | Expires / Max-Age | Size | HttpOnly | Secure | SameSite | Last Accessed | Partition Key JSESSIONID | 995C7FAE1C09CE07A6F2ABEDFFF38790 | localhost | / | Session | 42 | true | true | Lax | Mon, 03 Aug 2026 21:50:41 GMT |
Source	Dynamic Analysis - ZAP (A.5)

Criticality	Low
Description	When an exception is triggered, a debug error page is shown to the user, exposing the inner workings of the app such as the SQL commands that failed.
OWASP	A10:2025 Mishandling of Exceptional Conditions
CWE	CWE-209: Generation of Error Message Containing Sensitive Information
Location	Any endpoint that can trigger an exception such as the editPrompt and sendMessage endpoints.
Exploit	Generate an error such as by inputting 500 characters into the edit prompt content endpoint or the send message endpoint. See that an error page containing debug info is returned: http://localhost:8082/prompts/863/edit with a 500 character input: **Whitelabel Error Page** This application has no explicit mapping for /error, so you are seeing this as a fallback. Mon Aug 03 23:16:52 IST 2026 There was an unexpected error (type=Internal Server Error, status=500). could not execute statement [Data truncation: Data too long for column 'content' at row 1] [update prompt set category_id=?,content=?,created_by=?,owner=?,policy_flagged=?,title=?,updated_by=?,updated_on=?,visibility=? where id=?]; SQL [update prompt set category_id=?,content=?,created_by=?,owner=?,policy_flagged=?,title=?,updated_by=?,updated_on=?,visibility=? where id=?] org.springframework.dao.DataIntegrityViolationException: could not execute statement [Data truncation: Data too long for column 'content' at row 1] [update prompt set category_id=?,content=?,created_by=?,owner=?,policy_flagged=?,title=?,updated_by=?,updated_on=?,visibility=? where id=?]; SQL [update prompt set category_id=?,content=?,created_by=?,owner=?,policy_flagged=?,title=?,updated_by=?,updated_on=?,visibility=? where id=?] at org.springframework.orm.jpa.vendor.HibernateJpaDialect.convertHibernateAccessException(HibernateJpaDialect.java:297) at org.springframework.orm.jpa.vendor.HibernateJpaDialect.convertHibernateAccessException(HibernateJpaDialect.java:256) at org.springframework.orm.jpa.vendor.HibernateJpaDialect.translateExceptionIfPossible(HibernateJpaDialect.java:241) at org.springframework.orm.jpa.JpaTransactionManager.doCommit(JpaTransactionManager.java:567) http://localhost:8082/conversations/58 with a 500 character input: **Whitelabel Error Page** This application has no explicit mapping for /error, so you are seeing this as a fallback. Mon Aug 03 23:13:55 IST 2026 There was an unexpected error (type=Internal Server Error, status=500). could not execute statement [Data truncation: Data too long for column 'content' at row 1] [insert into conversation_message (content,conversation_id,created_on,policy_flagged,role) values (?,?,?,?,?)]; SQL [insert into conversation_message (content,conversation_id,created_on,policy_flagged,role) values (?,?,?,?,?)] org.springframework.dao.DataIntegrityViolationException: could not execute statement [Data truncation: Data too long for column 'content' at row 1] [insert into conversation_message
Recommended Fix	Create a generic error page with a generic error such as “Something went wrong” at src/main/resources/templates/error.html. SpringBoot will automatically use this as the error page. Log the error but don’t display debug messages to the user.
Source	Dynamic Analysis - ZAP (A.5)

Criticality	Low
Description	Passwords are checked using `.equals` which early returns as soon as character is detected to not match.
OWASP	Not OWASP Top 10
CWE	CWE-208: Observable Timing Discrepancy
Location	AuthService.java:23 `public Optional<User> login(LoginRequestDto request) { return userRepository.findByEmail(request.email()) .filter(u -> u.getPassword().equals(request.password())) .filter(User::isActive); }`
Exploit	By using a fuzzer and checking the response time difference between attacks, an attacker can detect timing discrepancies. When the request takes longer than before, the attacker knows they have learned one correct bit in the password and can build off that until they guess the whole password.
Recommended Fix	Passwords should be checked in constant time. Using Spring's DaoAuthenticationProvider with a password encoder such as BCryptPasswordEncoder means password hashes will be compared in constant time, removing this vulnerability.
Source	Manual Analysis

Vulnerability Findings - False Positives

Description	Source	Investigation
SQL Injection	Dynamic Analysis - ZAP (A.5)	Submitted inputs are stored as text and never executed as commands.
Path Traversal	Dynamic Analysis - ZAP (A.5)	All submitted inputs are stored in database fields, never submitted to filesystem calls or functions.
Cross-Site Scripting payloads stored in categories/prompts	AI Analysis (A.6, A.7)	All submitted values are displayed safely using thymeleaf using th:text and th:value, never treating unsafe html as safe and using as is.
Hardcoded passwords in DataSeeder.java	Static Analysis - Snyk (A.3)	This is intentional seed test data intentionally used for local testing.
Full User entity retained in HTTP session	AI Analysis (A.6, A.7)	The entire user object, including password, is currently stored in the session. As this is not sent to the client anywhere, CWE-201: Insertion of Sensitive Information Into Sent Data does not apply. However, it should be reduced to just include the user ID so an accidental session dump later does not lead to a vulnerability.

Conclusion

Several issues have been identified with PromptVault that must be fixed before a production release can be recommended. At minimum, upgrading vulnerable dependencies to new versions, implementing Spring Security, storing password hashes instead of as plaintext, fixing the identified broken access controls, and removing sensitive information from the logs is strongly recommended.

Appendix A

Snyk Results

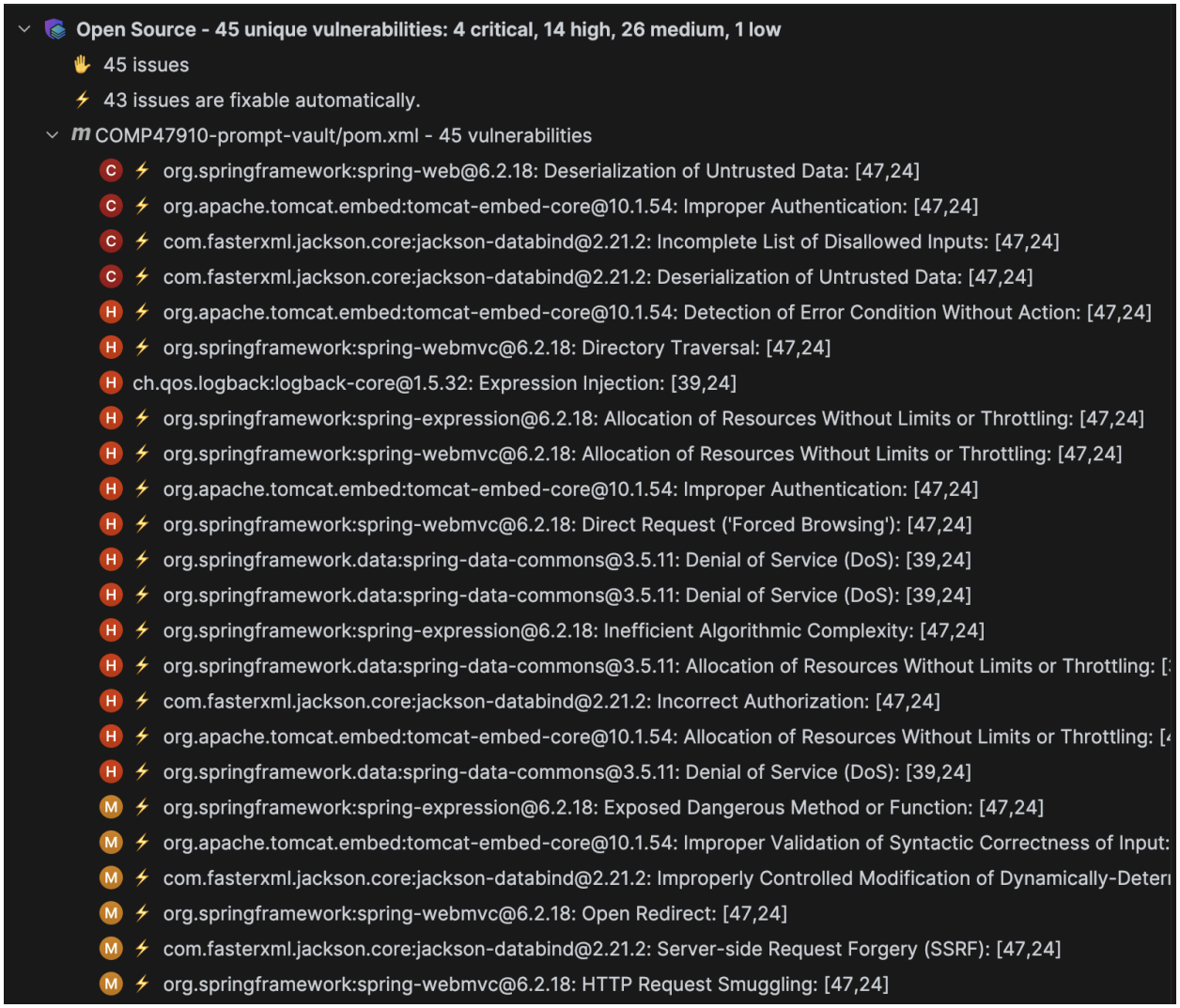


Image 1

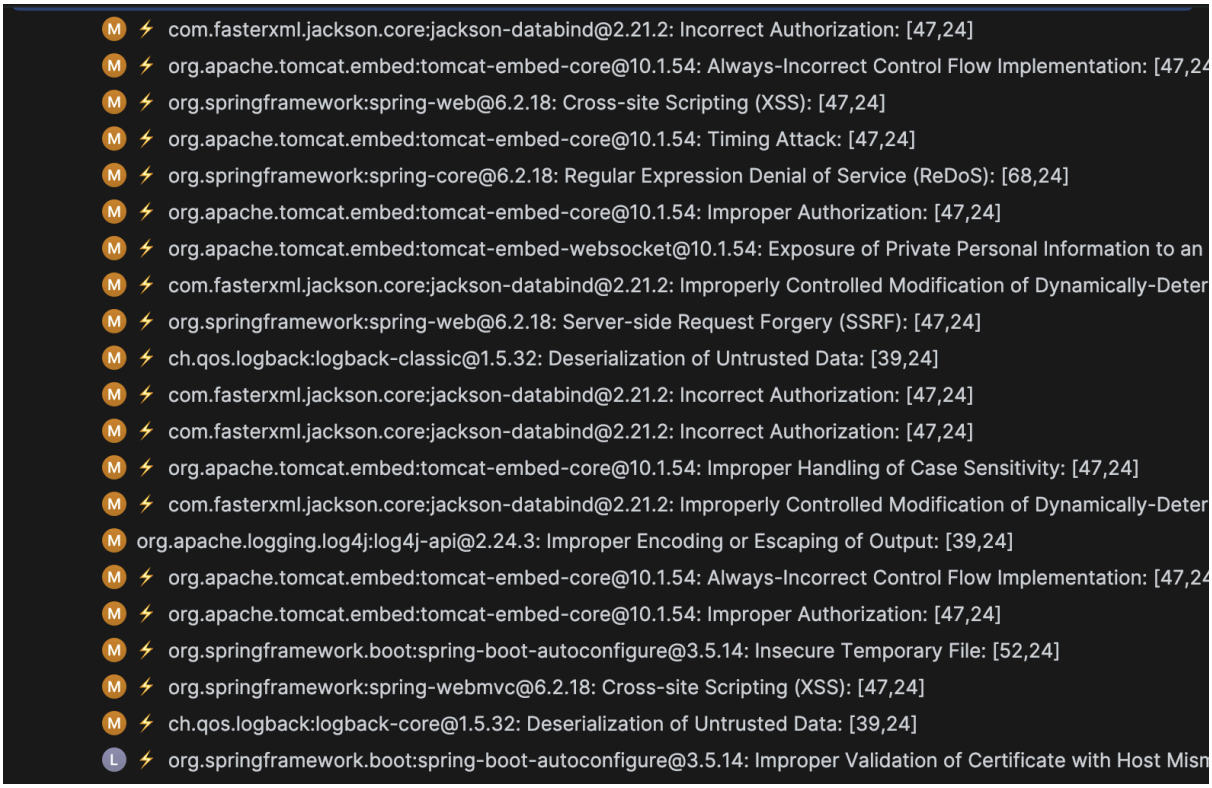


Image 2

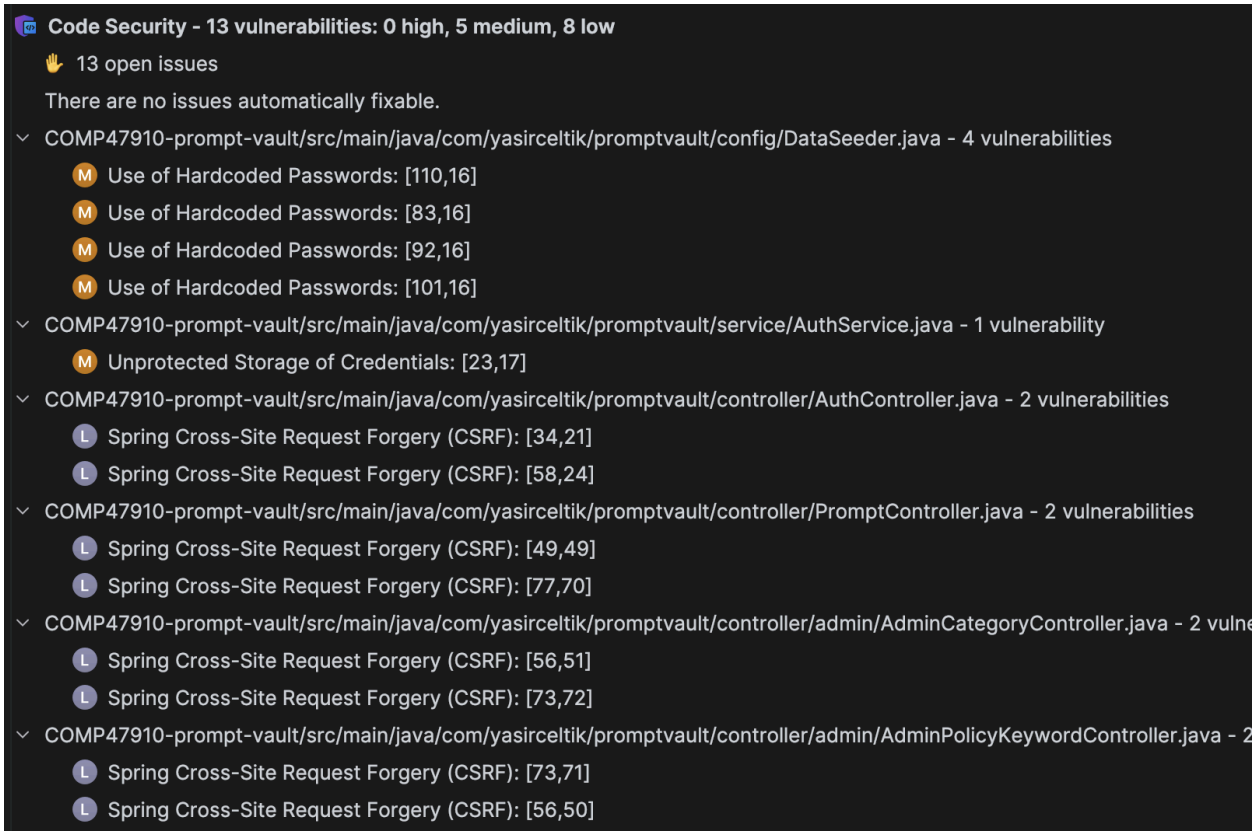


Image 3

OWASP Dependency Checker Results

How to read the report | Suppressing false positives | Getting Help: github issues

Project:

com.yasirceltik:promptvault:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 12.2.2
- *Report Generated On*: Tue, 21 Jul 2026 18:57:58 +0200
- *Dependencies Scanned*: 77 (52 unique)
- *Vulnerable Dependencies*: 4
- *Vulnerabilities Found*: 41
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
jackson-databind-2.21.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.21.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-databind:2.21.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.21.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.21.2	HIGH	7	Highest	41
log4j-api-2.24.3.jar	cpe:2.3:a:apache:log4j:2.24.3:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.24.3	MEDIUM	3	Highest	41
spring-core-6.2.18.jar	cpe:2.3:a:pivotal_software:spring_framework:6.2.18:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:6.2.18:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:6.2.18:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@6.2.18	CRITICAL	16	Highest	41
tomcat-embed-core-10.1.54.jar	cpe:2.3:a:apache:tomcat:10.1.54:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:10.1.54:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@10.1.54	CRITICAL	15	Highest	63

Image 4

ZAP Results

Image 5

ChatGPT GPT-5.6 Thinking - AI Analysis Results

Issue description	Location/File
Plaintext passwords are stored and compared directly, exposing all credentials if the database, backups, sessions, or heap dumps are accessed.	AuthService.java lines 21–24, 36–46; User.java lines 36–40
An authenticated user can edit another user's prompt by changing the prompt ID because editPrompt loads the prompt by ID and never checks its owner.	PromptController.java lines 63–84; PromptService.java lines 100–133; PromptRepository.java
An authenticated user can submit messages to another user's conversation because sendMessage loads the conversation by ID and ignores the supplied user for authorization.	ConversationController.java lines 58–65; ConversationService.java lines 75–117
A user can start a conversation using another user's private prompt ID because createConversation accepts any existing prompt without checking ownership or SHARED visibility.	ConversationController.java lines 29–36; ConversationService.java lines 53–70
The prompt edit page exposes another user's prompt data because it retrieves a prompt by ID without an ownership check before placing it in the model.	PromptController.java lines 63–73; PromptService.java lines 48–49
Ownership checks compare entity IDs using == or !=. Boxed Long values may be compared by object identity, causing inconsistent authorization decisions.	ConversationController.java lines 45–50; ConversationService.java lines 120–129; PromptService.java lines 136–143
Disabled users can remain authenticated because the full User entity is stored in the session and SessionService only checks whether it is present, not whether the database account is still active.	AuthController.java lines 39–48; SessionService.java lines 9–17; UserService.java lines 30–34
The complete User entity, including its password field and generated getter, is retained in the HTTP session, increasing credential exposure through session stores, serialization, or memory dumps.	AuthController.java line 47; SessionService.java lines 11–12; User.java lines 15–46
The loginAttempts field exists but is never incremented or checked, and AuthService has no throttling or lockout logic. Automated password guessing is therefore not restricted in the reviewed code.	User.java lines 48–50; AuthService.java lines 21–24
DTOs have no Bean Validation constraints, allowing null, blank, oversized, malformed, or weak values. Services immediately call methods such as toLowerCase(), which can also produce errors.	CreateCategoryDto.java; CreatePolicyKeywordDto.java; CreatePromptRequestDto.java; LoginRequestDto.java; RegisterRequestDto.java
An empty policy keyword matches every prompt and message because every Java string contains the empty string. A null keyword can trigger a NullPointerException.	CreatePolicyKeywordDto.java; PolicyKeywordService.java lines 32–46; ConversationService.java lines 79–83; PromptService.java lines 67–72
User-controlled prompt, message, category, keyword, username, and profile values are stored without sanitization. Stored XSS is possible if any Thymeleaf template renders them using th:utext or unsafe	PromptService.java; ConversationService.java; CategoryService.java; PolicyKeywordService.java; User.java

Image 6

Policy scanning loads every keyword and performs substring checks against unrestricted user content for every prompt edit/create and every message, allowing avoidable CPU/database consumption.	PromptService.java lines 61–72 and 100–112; ConversationService.java lines 75–83
User-controlled values and personal data are written to logs, including email addresses, prompt titles, category names, and policy keywords. Newline characters may also permit log-forging depending on the encoder.	AuthController.java lines 37–48; UserService.java lines 30–34; PromptService.java line 97; CategoryService.java lines 34–50; PolicyKeywordService.java lines 32–47
Administrative service methods have no method-level authorization and rely entirely on individual controllers remembering to perform manual role checks.	UserService.java; CategoryService.java; PolicyKeywordService.java; ConversationService.adminDeleteConversation(); PromptService administrative calls
An administrator can disable any account, including their own account or potentially the last active administrator, because setActive has no business-rule safeguards.	AdminUserController.java lines 55–60; UserService.java lines 30–34
The User role column is not marked nullable=false. A null role can create inconsistent authorization behavior or runtime failures in role comparisons.	User.java lines 42–46
Prompt titles are globally unique rather than unique per owner. One user can reserve common titles and prevent other users from creating prompts with them, producing a cross-user availability/business-logic issue.	Prompt.java lines 27–30; PromptRepository.java lines 15–18
Join entities allow null message, prompt, or policy references because their @ManyToOne/@JoinColumn mappings are not declared non-null. Invalid rows can undermine moderation records and cause failures.	MessagePolicyMatch.java lines 25–31; PromptPolicyMatch.java lines 25–31
CSRF protection cannot be established from the supplied files. Numerous state-changing POST endpoints would be vulnerable if Spring Security CSRF is disabled or tokens are absent.	All create/edit/delete/message POST controllers; Security configuration and Thymeleaf forms not supplied
Session fixation protection is not visible. The login method places the user into the existing session without explicitly rotating its ID.	AuthController.java lines 33–48; security configuration not supplied

Image 7